


UNIVERSITÀ DEGLI STUDI DI TORINO

**Dipartimento di Informatica**

Scuola di Scienze della Natura

**Corso di laurea triennale in Informatica**

Curricula 949351

Tesi di laurea triennale in informatica

# Panoramica delle catene di Markov parametriche e della loro applicazione ai processi decisionali di Markov parzialmente osservabili

**Relatore:**
Jeremy James Sproston

**Candidato:**
Paolo Alberto Bordis

**Sessione Novembre 2024**
a.a. 2023/2024

**Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.**

**Sommario**

Nel presente lavoro, si vuol andare a studiare la teoria delle catene di Markov parametriche.

Questo lavoro si propone di dimostrare un caso d'uso che le collega ai processi decisionali di Markov con osservazione parziale.

Viene approfondita la sezione sulla sintesi parametrica, con l'obiettivo di fornire una panoramica generale sul computabile in relazione ai modelli parametrici.

Viene inoltre analizzata una parte specifica legata a quest'ultima sezione, ovvero l'algoritmo usato per la computazione dei risultati.

Infine, procediamo con l'analisi dei dati utilizzando lo strumento Storm, identificando in particolare alcuni valori significativi.

Altri risultati, sebbene interessanti, risultano meno chiari.

# Indice


**1 Introduzione** **3**
1.1 Strumenti matematici . . . 3
1.2 Motivazione . . . 3

**2 Catene di Markov** **5**
2.1 Distribuzioni . . . 5
2.2 Discrete Time Markov Chains . . . 5
2.2.1 Cammini e Proprietà di un DTMC . . . 6
2.2.2 Raggiungibilità di un sottoinsieme di stati . . . 7
2.3 Markov Decision Process . . . 9
2.3.1 Cammini e Proprietà di un MDP . . . 9
2.3.2 Strategie . . . 10
2.3.3 Catena di Markov Indotta . . . 11
2.3.4 Specifiche e analisi quantitativa . . . 11
2.4 Parametric Markov Chain . . . 12
2.4.1 Definizione di pMC corretto o ben definito . . . 12
2.4.2 Istanziazione . . . 13
2.5 Partially Observable Markov Decision Process . . . 14
2.5.1 Cammini e strategie . . . 14
2.5.2 Finite State Controllers (FSCs) . . . 15

**3 Sintesi parametrica** **18**
3.1 Regione . . . 18
3.2 Problemi della sintesi parametrica . . . 19
3.3 Feasability problem . . . 20
3.3.1 Algoritmo di eliminazione degli stati . . . 21
3.4 Discesa del gradiente . . . 22
3.4.1 Introduzione . . . 22
3.4.2 Algoritmo Plain GD . . . 23

**4 Strumenti** **25**
4.1 Prism . . . 25
4.1.1 Sintassi . . . 25
4.1.2 Dichiarazione del modello . . . 25
4.1.3 Costrutti di base . . . 26
4.1.4 Principali modelli analizzati . . . 27
4.2 Storm . . . 31
4.2.1 Storm CLI e Stormpy . . . 31

4.2.2 Motori . . . . . 31
4.2.3 Script per l'Analisi Automatica con Storm. . . . . 32

**5 Sperimentazione e Conclusioni** **38**
5.1 Casi di studio . . . . . 38
5.2 Conclusioni finali . . . . . 42

# Capitolo 1

# Introduzione

Il Model Checking è un metodo ampiamente utilizzato nella verifica formale di sistemi complessi, in cui si valutano algoritmicamente le proprietà di sicurezza e di correttezza del sistema.

Un esempio di impiego di questa metodologia risiede nell'esaminare il comportamento dei sistemi di guida autonoma rispetto a specifiche di sicurezza predefinite [2].

L'obiettivo è garantire che le azioni e le decisioni prese dal veicolo rispettino questi requisiti, contribuendo così a ridurre il rischio di incidenti.

## 1.1 Strumenti matematici

Nel mondo del Model Checking vengono impiegati numerosi strumenti matematici, poiché godono di proprietà già dimostrate teoricamente, rendendoli strumenti affidabili.

Nel corso della trattazione, si analizzano prevalentemente le Catene di Markov nella loro forma, anche più avanzata.

Le Catene di Markov sono strumenti matematici impiegati per risolvere numerosi problemi in diversi campi.

Il modello contiene stati e transizioni; semanticamente, la nozione di stato può rappresentare l'avanzamento del lavoro di un processo o l'evoluzione di un sistema.

Grazie alle sue proprietà, la Catena di Markov si rivela un modello adattabile alla modellazione del problema.

## 1.2 Motivazione

Lo studio condotto si concentra su due modelli significativi del Model Checking di sistemi probabilistici: il modello parametrico e il modello con osservazione parziale.

In questo lavoro, desideriamo esaminare la relazione tra questi due formalismi, utilizzando i modelli parametrici per analizzare i modelli di osservazione parziale e trarre valutazioni in termini di prestazioni.

L'analisi viene condotta attraverso l'utilizzo dello strumento Storm Model Checker [9], progettato specificamente per effettuare analisi su entrambi i modelli.

Questo strumento ha anche applicazioni pratiche, poiché viene impiegato all'interno di un software di verifica chiamato Safest [8].

# Capitolo 2

# Catene di Markov

## 2.1 Distribuzioni

Introduciamo un concetto di base che è la distribuzione probabilistica. Una distribuzione probabilistica su un insieme $Q$ numerabile è una funzione $\mu : Q \to [0, 1]$ dove $\sum_{q \in Q} \mu(q) = 1$.

Diciamo infine che per Dist($Q$) intendiamo l'insieme di distribuzioni su $Q$ e definiamo la notazione $\{q_1 \mapsto \frac{1}{3}, q_2 \mapsto \frac{2}{3}\}$, denota ad esempio che $q_1$ occorre con probabilità $\frac{1}{3}$.
Definiamo anche il concetto di *supporto* alla distribuzione o $supp(\mu) = \{q \in Q \mid \mu(q) > 0\}$, che definisce l'insieme degli elementi in $Q$ tali che sono stati assegnati dei valori di probabilità positivi.

Specifiamo infine il concetto di *spazio di probabilità* definito con una tripla $(\Omega, \mathcal{F}, Pr)$ dove $\mathcal{F} \subseteq 2^{\Omega}$ è una $\sigma$-algebra su $\Omega$ dove:

- $\emptyset, \Omega \in \mathcal{F}$;
- se $A \in \mathcal{F}$, allora $\Omega \setminus A \in \mathcal{F}$;
- se $A_i \in F$ per tutti gli $i \geq 0$, allora $\cup_{i \geq 0} A_i \in \mathcal{F}$

e $Pr : \mathcal{F} \to [0, 1]$ viene nominata *misura di probabilità* dove:

- $Pr(\emptyset) = 0$ e $Pr(\Omega) = 1$
- $Pr(\cup_{i \geq 0} A_i) = \sum_{i \geq 0} Pr(A_i)$ per tutte le coppie partizioni di sequenze disgiunte $A_1, A_2, ... \in \mathcal{F}$

## 2.2 Discrete Time Markov Chains

Introduciamo adesso formalmente il concetto di catena di Markov a tempo discreto che rappresenta lo strumento alla base del mio elaborato.

Definiamo una catena di Markov a tempo discreto (DTMC) come una tupla di quattro elementi $\langle S, s_{init}, P, L \rangle$, dove $S$ rappresenta l'insieme di tutti gli stati possibili o anche chiamato spazio degli stati, $s_{init} \in S$ rappresenta lo stato iniziale della DTMC.

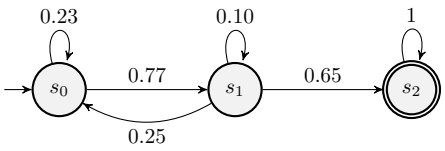
Specifichiamo ancora $P : S \times S \to [0, 1]$ come una funzione che, dato una coppia di stati $(s, s')$, con $s, s' \in S$, assegna un valore nell'intervallo $[0, 1]$. Inoltre, $\sum_{s' \in S} P(s, s') = 1$ per tutti gli $s \in S$.

L'ultimo elemento della tupla definisce la parte letterale dove possiamo etichettare gli stati dandone un significato semantico. Più in generale, $L : S \to 2^{AP}$, ovvero dato uno stato $s$, associato $L(s)$ viene mappato nell'insieme delle preposizioni atomiche AP.

Questo tipo di insieme è generalmente impiegato nell'identificazione di quegli stati che semanticamente vengono definiti come stati di *error* o *idle*, ovvero dove il comportamento del sistema comporta un errore oppure dove il sistema si dice non attivo.

La $P$ può essere rappresentata in formato matriciale dove le righe e le colonne rappresentano tutti gli stati della DTMC.

Per esempio, con questa matrice stiamo definendo un modello che rispecchia la formulazione a grafo:




$$\mathcal{P} = \begin{bmatrix} 0.23 & 0.77 & 0 \\ 0.25 & 0.10 & 0.65 \\ 0 & 0 & 1 \end{bmatrix}$$

Figura 2.1: Esempio di DTMC con la sua matrice di transizione.

Ipotizzando una DTMC come in Figura 2.1, notiamo come la sua matrice di transizione identifica i suoi valori di probabilità per ogni stato, nello specifico possiamo notare ad esempio rispetto allo stato $s_0$ abbiamo: $\mathcal{P}(s_0, s_0) = 0.23$, $\mathcal{P}(s_0, s_1) = 0.77$, $\mathcal{P}(s_0, s_2) = 0$.

### 2.2.1 Cammini e Proprietà di un DTMC

Stabilito che cos'è una DTMC non ci resta che soffermarci sulla nozione di cammino finito e infinito. Supponiamo di avere un DTMC $D = \langle S, s_{init}, P, L \rangle$, possiamo definire $\pi = s_0 \to s_1 \to s_2 \to ...$ dove $P(s_i, s_{i+1}) > 0$ per tutti gli i $\in \mathbb{N}$.

Ovvero stiamo precisando che un path è come una concatenazione di stati in cui il DTMC commuta senza terminare mai di fatto.

Determiniamo ora l'insieme di tutte queste sequenze infinite con l'insieme $\Pi_{inf}$ e diciamo che se precedentemente $\pi$ era una sequenza infinita allora $\pi \in \Pi_{inf}$.

Procediamo con la nozione di sequenza finita $\pi = s_0 \to s_1 \to s_2 \to ... \to s_n$ dove $P(s_i, s_{i+1}) > 0$ per tutti gli $0 \leq$ i $<$ n, che termina e il suo stato finale è $s_n$ e l'insieme di tutte queste sequenze lo definiamo come $\Pi_{fin}$. È opportuno

definire $\Pi = \Pi_{fin} \cup \Pi_{inf}$, l'insieme di tutti i possibili cammini finiti e infiniti. Questa restrizione insiemistica ci permette di discutere sul comportamento di un cammino rispetto al modello che stiamo analizzando.

Introdotte le nozioni di cammini, vogliamo ora presentare delle proprietà di quest'ultimi, quindi per tutti i $\pi \in \Pi_{fin}$ abbiamo che $last(s_0 \to s_1 \to s_2 \to .. \to s_n) = s_n$, ovvero se consideriamo un cammino finito allora il suo stato finale sarà $s_n$. A questo punto diciamo che la *lunghezza* di una sequenza finita è il numero di passi necessari per ottenere come stato finale $s_n$ che sarebbe $|\pi| = n$, con $\pi \in \Pi_{fin}$.

L'ultima proprietà che andrò a sviluppare è legata a qualsiasi cammino $\pi \in \Pi$, ovvero il $first(s_0 \to s_1 \to s_2 \to .. \to s_n) = s_0$ che contrariamente al $last$ restituisce lo stato di partenza del cammino.

### 2.2.2 Raggiungibilità di un sottoinsieme di stati

Noi spesso considereremo insieme di cammini che partono da uno certo tipo di sottoinsiemi di stati fissato. Consideriamo uno stato $s \in S$ ovvero gli stati che noi specifichiamo come iniziali da cui viene generato il cammino.

Procediamo con

$$\Pi^D(s) = \{\ \pi \in \Pi^D \mid first(\pi) = s\ \}$$

che contiene l'insieme di tutti i possibili path finiti e infiniti tali che il loro stato iniziale risiede in $S'$.

Ora invece definiamo

$$\hat{\Pi}^D(\Delta) = \{\ \pi \in \Pi^D_{fin} \mid last(\pi) \in \Delta\ \}$$

tali che sono finiti e il loro stato finale appartiene a questo sottoinsieme chiamato $\Delta$.

Concludiamo derivando

$$\Pi^D_{fin}(\Delta, s) = \Pi^D(s) \cap \hat{\Pi}^D(\Delta)$$

ossia l'insieme dato dall'intersezioni dei due precedenti che ci resituisce l'insieme dei path dove abbiamo s come stato iniziale e un insieme di possibili stati $s' \in \Delta$ come stato finale.

A questo punto definiamo in maniera più semplice il predicato $Reach : S \to 2^S$, definiti come $Reach(s) = \{\ t \in S \mid \Pi^M_{fin}(\{t\}, s) \neq \emptyset\ \}$ ovvero l'insieme degli stati raggiungibili a partire da uno stato $s$ dato.

Consideriamo quattro cammini possibili:

$$\pi_0 : s_0 \to s_1 \to s_2 \to \cdots \to s_8$$
$$\pi_1 : s_0 \to s_3 \to s_4 \to \cdots \to s_9$$
$$\pi_2 : s_3 \to s_5 \to s_6 \to \cdots \to s_9$$
$$\pi_3 : s_0 \to s_3 \to s_4 \to \dots$$

vogliamo andare a capire quali stati sono raggiungibili da $s_0$, ovvero vogliamo calcolare $Reach(s_0)$.
Notiamo che abbiamo bisogno di derivare i due insiemi, il primo $\Pi^D(s_0)$ , ovvero l'insieme dei cammini tali che hanno come first proprio $s_0$, in particolare abbiamo $\pi_0, \pi_1, \pi_3 \in \Pi^D(s_0)$.
Il secondo insieme invece è $\hat{\Pi}^D(\Delta)$ che determina l'insieme dei cammini finiti.
Determiniamo l'intersezione dei due, e troviamo che i percorsi che soddisfano questa congiunzione sono proprio $\pi_0, \pi_1 \in \Pi$, da questo concludiamo che $Reach(s_0) = \{s_8, s_9\}$.

Approfondiamo ora la nozione di raggiungibilità, in particolare siamo interessati a legare una stima di probabilità all'insieme dei path che noi identifinchiamo come interessanti, perchè giungono a degli stati definiti raggiungibili.

Sia data la DTMC D $\langle S, s_{init}, P, L\rangle$, definiamo la funzione $P : \Pi^M_{fin}(s) \to [0, 1]$.

Poniamo ora le basi alla costruzioni del probabilità legata a quei path finiti:

$$P(\pi) = \prod_{i=0}^{n-1} P(s_i, s_{i+1}).$$

che definisce la *massa* di $\pi$, ovvero il prodotto di tutte le transizioni.

Iniziamo defininendo, il *basic cylinder* $C_\pi$, che consiste nell'insieme di tutti i percorsi infiniti che partono da $\pi$.

Inoltre usando le proprietà del basic cylinders, possiamo costruire lo spazio di probabilità $(\Pi^{D,s}, \mathcal{F}_{D,s}, Pr^{D,s})$, dove $\mathcal{F}_{D,s}$ è una $\sigma$-algebra generata dal basic cylinder $\{C_\pi \mid \pi \in \Pi^{D,s}_{fin}\}$ e $Pr^{D,s}$ è l'unica misura dove

$$Pr^{D,s}(C_\pi) = P(\pi) \text{ per tutti i } \pi \in \Pi^{D,s}_{fin}.$$

## 2.3 Markov Decision Process

Introduciamo ora gli MDP. Un MDP $M$ è una tupla $\langle S, s_{init}, Act, \mathcal{P} \rangle$ dove abbiamo due nuovi elementi da definire, il primo è $Act$, che rappresenta l'insieme di tutte le azioni possibili.

Le possibili azioni vengono usate per ridefinire la nostra funzione parziale di transizione probabilistica $\mathcal{P} : S \times Act \to Distr(S)$.

Inoltre, definiamo l'insieme $\mathbb{A}ct(s) = \{\alpha \in Act \mid \mathcal{P}(s, \alpha) \neq \bot\}$ come l'insieme delle azioni possibili per lo stato $s$.

Assumiamo inoltre che $\mathbb{A}ct(s) \geq 1$ per tutti gli stati $s$. Questo garantisce che per ogni stato ci sia almeno un'azione ben definita.

Ora possiamo ridefinire la funzione di transizione probabilistica. Dato $s, s' \in S$, e $\alpha \in Act(s)$, abbiamo $\mathcal{P}(s, \alpha)(s')$, ovvero, dato $s$ e scegliendo $\alpha$ come azione, si ottiene uno stato $s'$.

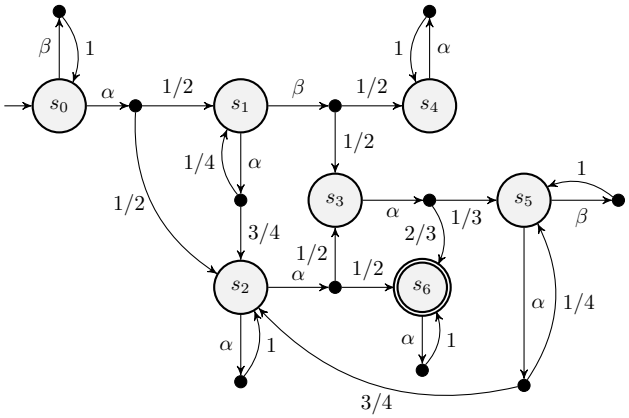



Figura 2.2: Esempio di MDP.

Per esempio, rispetto alla Figura 2.2, abbiamo:

$$\mathcal{P}(s_1, \beta) = \{s_4 \mapsto \frac{1}{2}, s_3 \mapsto \frac{1}{2}\}$$

Questa mappatura ci indica che, grazie all'azione $\beta$, quando ci troviamo nello stato $s_1$, possiamo finire con probabilità $\frac{1}{2}$ in $s_3$ o con probabilità $\frac{1}{2}$ in $s_4$.
Oppure, rispetto all'azione $\alpha$, abbiamo:

$$\mathcal{P}(s_3, \alpha) = \{s_5 \mapsto \frac{1}{3}, s_6 \mapsto \frac{2}{3}\}$$

### 2.3.1 Cammini e Proprietà di un MDP

Una volta stabilito il modello MDP, non ci resta che soffermarci sulla nozione di *path*, ovviamente tratteremo principalmente cammini finiti ma possiamo anche

estendere la trattazione ai cammini infiniti.

Sia dato un cammino finito $\pi = s_0 \xrightarrow{\alpha_0} s_1 \xrightarrow{\alpha_1} s_2 \xrightarrow{\alpha_2} ... \xrightarrow{\alpha_{n-1}} s_n$ dove $\alpha_i \in Act$ e per ogni $0 \leq i < n$, dobbiamo avere $\mathcal{P}(s_i, \alpha_i)(s_{i+1}) > 0$. In più, $s_i \in S$ per ogni $0 \leq i \leq n$.

Introduciamo adesso gli insiemi di path finiti e infiniti, in particolare se $\pi$ era finito allora $\pi \in \Pi^M_{fin}$. Invcece dato il cammino infinito $\pi = s_0 \xrightarrow{\alpha_0} s_1 \xrightarrow{\alpha_1} s_2 \xrightarrow{\alpha_2} ... \xrightarrow{\alpha_{n-1}} ...$ possiamo dire che $\pi \in \Pi^M_{inf}$.

Procediamo definendo la *lunghezza* del cammino, ovvero $|\pi| = n$ esattamente come lo abbiamo definito in sottosezione 2.2.1.

### 2.3.2 Strategie

Definendo le azioni per gli MDP, abbiamo aumentato il non-determinismo, cioè abbiamo introdotto la possibilità di effettuare scelte rispetto a un cammino dato. Tuttavia, per determinare il valore di probabilità di un insieme di cammini, è necessario risolvere questo tipo di scelta, motivo per cui vengono implementate le strategie.

Vogliamo quindi definire *strategia* per un MDP $M$ come una funzione $\sigma : \Pi^M_{fin} \to Distr(Act)$, dove il $supp(\sigma(\pi)) \subseteq \mathbb{A}ct(last(\pi))$ per tutti i $\pi \in \Pi^M_{fin}$.

Successivamente esaminiamo due nozioni fondamentali:

- $last(\pi) = last(\pi') \implies \sigma(\pi) = \sigma(\pi')\ \forall \pi, \pi' \in \Pi^M_{fin}$, specifica che una strategia è *senza memoria*
- $|supp(\sigma(\pi))| = 1$ per tutti i $\pi \in \Pi^M_{fin}$, definisce che la strategia è *deterministica*

Denotiamo l'insieme di tutte le strategie senza memoria e deterministiche come $\Sigma^M$.

Useremo per identificare strategie deterministiche la funzione definita come $\sigma : \Pi^{\mathcal{M}}_{fin} \to Act$, e $\sigma : S \to Dist(Act)$ che indentifica una strategia senza memoria.

Riconsideriamo l'MDP in Figura 2.2 e cerchiamo di definire delle strategie più chiare come esempi per comprendere meglio il significato di strategia deterministica.
Ad esempio siano date delle strategie:

$$\sigma_1(\pi) = \{s_1 \mapsto \{\alpha \mapsto 0, \beta \mapsto 1\}, s_3 \mapsto \alpha\}$$

La strategia presentata è senza memoria ma non deterministica.

$$\sigma_2(\pi) = \{s_1 \mapsto \beta, s_3 \mapsto \alpha\}$$

La strategia $\sigma_2$ inveve è sia deterministica che senza memoria.

### 2.3.3 Catena di Markov Indotta

Poniamo ora l'attenzione sulla nozione che, sulla base delle definizioni precedenti di strategie e azioni, permette di generare una nuova catena di Markov, chiamata *Indotta*.

Sia dato un *MDP* $M$ e una strategia $\sigma \in \Sigma^M$. Chiamiamo la catena di Markov indotta (MC Indotta) come $M[\sigma] = (\Pi^M_{fin}, s_{init}, \mathcal{P}[\sigma])$, dove:

$$\mathcal{P}[\sigma](\pi, \pi') := \begin{cases} \mathcal{P}(last(\pi), \alpha, s') \cdot \sigma(\pi)(\alpha) & \text{se } \pi' = \pi \xrightarrow{\alpha} s' \\ 0 & \text{altrimenti.} \end{cases}$$

La generazione di questo nuovo modello tiene conto solo delle transizioni nel modello che sono raggiungibili attraverso una strategia. Questo porta ad annullare, assegnando una probabilità pari a 0, tutte le transizioni che non vengono considerate dalla strategia.

### 2.3.4 Specifiche e analisi quantitativa

L'analisi quantitativa è un tipo di analisi che ci restituisce al posto del valore esatto, un valore approssimato rispetto ad una specifica proprietà che si vuole andare a verificare.

Vogliamo ora soffermarci sulle specifiche che sono delle notazioni che vengono formalizzate a livello teorico e ci permettono di andare a capire bene il concetto di *proprietà*.

In particolare ogni riferimento anche se teorico rispecchia chiaramente la sintassi e quindi quello che letteralmente si andrebbe a scrivere all'interno di uno scritpt per l'analisi sul calcolatore[1].

Dato un MDP $M$, un $G \subseteq S$ che definiamo come stati *Goal* , un sottoinsieme $B \subseteq S$ chiamati stati di *Bad* e threshold $\lambda \in [0, 1)$ possiamo definire la specifica

$$\mathbb{P}_{>\lambda}(\neg B \; U \; G)$$

Questa definizione ci permette di discutere se una certa specifica è soddisfatta per una strategia $\sigma \in \Sigma^{\mathcal{M}}$.

In particolare vogliamo andare a calcolare la probabilità $Pr^{\mathcal{M}^\sigma}(\neg B \; U \; G)$, ovvero la raggiunbilità degli stati Goal senza passare da quelli Bad e con probabilità maggiore del threshold $\lambda$.

In maniera analoga si può valutare anche il caso complementare, ovvero:

$$\mathbb{P}_{<\lambda}(\neg B \; U \; G)$$

che in questo vincola a trovare un certo valore che si trovare al di sotto di $\lambda$.

Notiamo infine che la notazione usata ha di per sè una sua sintassi che eredita dalla Logica PCTL[2], in particolare possiamo dire che $\neg B$ si riferisce al fatto di lasciare stare gli stati contenuti in Bad, mentre $U$ sarebbe l'*Until*, ovvero $\neg B \; U \; G$ è soddisfatta in un qualche punto del cammino e in tutti gli stati precedenti $\neg B$ è vera.

[1] Nella sottosezione 4.2.3 è presente uno script di implementazione.

[2] Estensione della Logica Temporale che viene usata per esprimere proprietà nelle catene di Markov.

## 2.4 Parametric Markov Chain

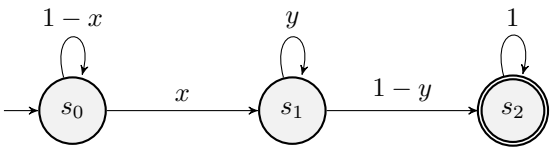
Introduciamo ora la nozione di catena di Markov parametrica, che rappresenta la parte teorica centrale di questo elaborato.

Consideriamo quindi $\mathcal{T}$, una $pMC$, come una tupla $\langle S, s_{init}, \mathcal{X}, \mathbf{P}\rangle$, dove $S$ è l'insieme finito di stati, $s_{init}$ è lo stato iniziale, $\mathcal{X}$ rappresenta l'insieme finito di tutti i parametri, e $\mathbf{P} : S \times S \to \mathbb{Q}(\mathcal{X})$ è la funzione di transizione parametrica.

$\mathbb{Q}(\mathcal{X})$ rappresenta l'insieme di tutte le possibili funzioni razionali definite sulla base di $\mathcal{X}$.

Le funzioni appartenenti a questo insieme sono del tipo $\frac{f}{g}$, dove $g \neq 0$.

Esempi di funzioni razionali sono $\frac{y^7}{x^3}$ o $\frac{x^4}{1-y^2}$.




$$\mathbf{P} = \begin{bmatrix} 1-x & x & 0 \\ 0 & y & 1-y \\ 0 & 0 & 1 \end{bmatrix}$$

Figura 2.3: Esempio di $pMC$ con la sua matrice di transizione.

Dalla Figura 2.3, possiamo notare che l'attenzione è posta sui parametri che si trovano sugli archi, i quali non hanno valori reali come negli altri esempi.

Inoltre, anche la matrice di transizione è diversa, poiché contiene funzioni razionali dipendenti da variabili. La particolarità di questo modello matematico risiede nella possibilità di considerare un intervallo di valori per ogni parametro.

Questa osservazione, tutt'altro che banale, ci consente di analizzare non solo singole istanze di parametri, ma anche di esplorare eventuali correlazioni tra parametri diversi.

### 2.4.1 Definizione di pMC corretto o ben definito

La definizione seguente ci consente di stabilire la correttezza di un pMC, la quale è legata alla valutazione appropriata dei parametri.

In particolare, possiamo affermare che non tutte le valutazioni effettuate sui parametri di un pMC rendono significativo il contesto; per questo motivo ci occupiamo di definire questo sottoinsieme.

Quindi, dato $\mathcal{M}$, un pMC con parametri $\mathcal{X}$, possiamo affermare che una valutazione $val : \mathcal{X} \to \mathbb{R}$ di $\mathcal{X}$ è ben definita per $\mathcal{M}$ se:

- le probabilità non sono negative, cioè

$$\mathbf{P}(s, s')[\text{val}] \geq 0, \quad \text{per tutti } s, s' \in S.$$

- la somma di tutte le transizioni uscenti rispetto ad uno stato dato è pari a 1,

$$\sum_{s' \in S} \mathbf{P}(s, s')[\text{val}] = 1, \quad \text{per tutti } s \in S.$$

Possiamo quindi definire $Val^{wd}_{\mathcal{M}}$ come l'insieme di tutte le valutazioni ben definite per $\mathcal{M}$.

Ora possiamo dare un esempio concreto di questa proprietà, consideriamo il pMC in Figura 2.3 e cerchiamo di trovare una valutazione corretta rispetto a quanto definito precedentemente.
Rispetto a questo esempio possiamo stabilire l'insieme:

$$Val^{wd}_{M} = \{val \in Val \mid 0 \leq val(x),\ val(y) \leq 1\}.$$

Possiamo dire che una possibile valutazione non è

$$\{x \to 2, y \to 0\},$$

in quanto non soddisfa il primo punto della definizione di Val ben definita, perchè rispetto al primo parametro, abbiamo $1 - x[val] < 0$.
Una possibile valutazione potrebbe essere

$$\{x \to 0.3, y \to 0.1\}.$$

### 2.4.2 Istanziazione

Sia dato $\mathcal{M}$ $pMC$ e una valutazione $val \in Val^{wd}_{M}$, diciamo che *l'istanziazione* di $\mathcal{M}$ è una catena di Markov $M[val] := \langle S, s_{init}, \mathcal{P} \rangle$ con:

$$\mathcal{P}(s, s') := \mathbf{P}(s, s')[val] \quad \text{per tutti gli } s, s' \in S.$$

L'istanziazione, quindi, rispetto alla valutazione genera una DTMC. In questa nuova DTMC, le transizioni etichettate precedentemente da parametri vengono rimpiazzate con le valutazioni assegnate ai singoli parametri.

## 2.5 Partially Observable Markov Decision Process

Concludiamo con l'ultimo modello analizzato, la *catena di Markov con osservazione parziale*, chiamata POMDP.

Questo modello è caratterizzato, come vedremo, dal concetto di *osservazione*, che si lega agli stati del modello e ci permette di analizzare situazioni in cui il sistema deve tener conto dell'incertezza rispetto alle scelte effettuate.

Consideriamo quindi *POMDP* come una tupla $\langle M, Z, O \rangle$ dove la prima componente $M$ si riferisce a un classico MDP, come descritto nella sezione sezione 2.3.

Le ultime due componenti sono: $Z$, che contiene l'insieme di tutti gli stati osservati, e $O : S \to Z$, che si riferisce alla funzione di osservazione.

Possiamo dire che la funzione di osservazione lega rispetto alle azioni, in quanto non è possibile avere strategie diverse per due stati distinti, partendo dall'assunto che entrambi abbiano la stessa osservazione.

In altre parole, possiamo affermare che:

$$O(s) = O(s') \implies Act(s) = Act(s') \; \forall s, s' \in S$$

Seguendo l'esempio in Figura 2.4, possiamo notare che abbiamo la stessa colorazione per gli stati $s_2$, $s_3$ e $s_4$. Ciò significa che abbiamo $O(s_2) = O(s_3) = O(s_4)$, ovvero ci sono le stesse osservazioni per tre stati diversi.

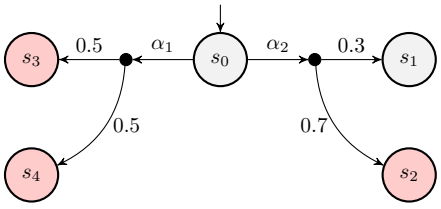



Figura 2.4: Esempio di POMDP.

### 2.5.1 Cammini e strategie

Procediamo con la formalizzazione del concetto di cammino o path di un POMDP, caratterizzato non solo dalle azioni intraprese ma anche dalle osservazioni lungo il cammino.

Sia dato un POMDP $\mathcal{V}$; definiamo $\Pi^{\mathcal{V}}_{fin}$ come l'insieme di tutti i cammini finiti possibili. Se abbiamo $\pi = s_0 \overset{\alpha_0}{\to} s_1 \overset{\alpha_1}{\to} \dots \overset{\alpha_{n-1}}{\to} s_n$, diciamo che questo termina, e dunque $\pi \in \Pi^{\mathcal{V}}_{fin}$.

Inoltre, associamo a ciascun cammino la sua sequenza di osservazioni e azioni $O(\pi) = O(s_0) \overset{\alpha_0}{\to} O(s_1) \overset{\alpha_1}{\to} \dots \overset{\alpha_{n-1}}{\to} O(s_n)$.

Ora, rimane solo da definire la nozione di strategia, che in questo caso è basata sulle osservazioni. Diciamo strategia basata sull'osservazione una strategia $\sigma$ per un POMDP tale che $\sigma(\pi) = \sigma(\pi')$ per tutti i $\pi, \pi' \in \Pi^{\mathcal{V}}_{fin}$ con $O(\pi) = O(\pi')$.

### 2.5.2 Finite State Controllers (FSCs)

Data la definizione di strategia basata su osservazioni, possiamo determinare FSC o *Finite State Controller* ovvero una strategia basata sulle osservazioni con memoria finita.

In generale la struttura di un FSC è quello di una tupla $\langle N, n_1, \gamma, \delta\rangle$, dove $N$ è il numero di nodi, $n_1$ il nodo iniziale, $\gamma : N \times Z \to Distr(Act)$ definita come una funzione di mappatura per le azioni e $\delta : N \times Z \times Act \to Distr(N)$ come una funzione di *update* rispetto alla memoria.

Ogni FSC genera una strategia con memoria finita.

Dato un nodo $n$ e un osservazione z rispetto allo stato corrente del POMDP, la prossima azione $\alpha$ è scelta causualemente secondo la distribuzione $\gamma(n, z)$. Il nodo successivo quindi del FSC è determinato casualmente da $\delta(n, z, \alpha)$.

Un FSC con k nodi memoria viene chiamato un *k-FSC*. Se $k = 1$, un FSC descrive una strategia senza memoria.

Ora invece dobbiamo determinare un pMC andando combinare il k-FSC e il POMDP, per determinarlo abbiamo bisogno della nozione di Remain o *residuo*, in particolare definita come $Remain : Z \to Act$, dove $Remain(z) \in Act(z)$ che sarebbe l'azione per cui viene assegnato il residuo di probabilità dopo aver precedentemente assegnato ad ogni azione di $A(z)$ una probabilità.

Determinamo quindi la pMC *Indotta*. Sia data $\mathcal{R} = \langle M, Z, O\rangle$,
M=$\langle S, s_{init}, Act, \mathcal{P}\rangle$ e $k > 0$ nodi di memoria.

La pMC *Indotta* è definita come una tupla $\langle S_{M,k}, S_{init,k}, V_{M,k}, Prob_{M,k}\rangle$ dove:

$S_{M,k} : S \times \{0, ..., k-1\}$, insieme di coppie formate dagli stati del MDP e dai nodi

$S_{init,k} = \langle s_{init}, 0\rangle$, coppia iniziale che sarà formata dallo stato iniziale del MDP e dal nodo 0

$V_{M,k} = \{p^{z,n}_{\alpha} z \in Z, n \in \{0, ..., k-1\}, \alpha \in Act(z), \alpha \neq Remain(z)\}$
$\cup \{q^{z,n}_{\alpha,n'} z \in Z, n, n' \in \{0, ..., k-1\}, n' \neq k-1, \alpha \in A(z)\}$

$Prob_{M,k} = \sum_{\alpha \in Act(s)} H(s, s', \alpha)$ per tutti $s, s' \in S'$

dove $H : S_{M,k} \times S_{M,k} \times Act \to \mathbb{R}$ è definita come:

$$H(\langle s, n\rangle, \langle s', n'\rangle, \alpha) = \mathcal{P}(s, \alpha, s') \cdot \left\{ \begin{array}{ll} p^{z,n}_{\alpha} & \text{se } \alpha \neq \text{Remain}(z) \\ 1 - \sum_{\beta \neq \alpha} p^{z,n}_{\beta} & \text{se } \alpha = \text{Remain}(z) \end{array} \right\} \cdot \left\{ \begin{array}{ll} q^{z,n}_{\alpha} & \text{se } n' \neq k-1 \\ 1 - \sum_{\bar{n} \neq n'} q^{z,n}_{\alpha,\bar{n}} & \text{se } n' = k-1 \end{array} \right\}.$$

Intuitivamente possiamo dire che la funzione $H(s, s', \alpha)$ descrive la probabilità di passare dallo stato $s$ a $s'$ scegliendo come azione $\alpha$ nella pMC Indotta.

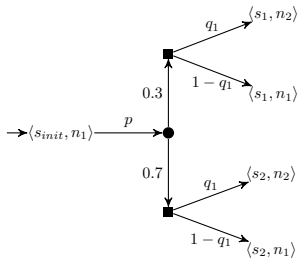



Figura 2.5: Esempio di pMC Indotta.

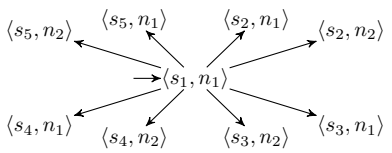



Figura 2.6: Esempio di $MC$ indotta $\mathcal{M}^{\sigma\,\mathcal{A}}$.

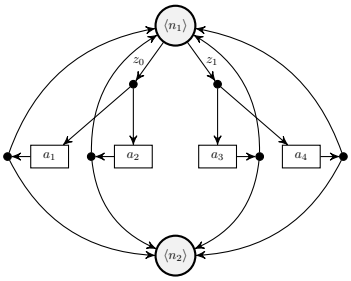



Figura 2.7: Esempio di $FSC$ con due nodi di memoria.

Ora forniremo un esempio di quanto abbiamo formalizzato, andando a capire le varie nozioni introdotte.
In particolare possiamo vedere come in Figura 2.7 vengono definite le transizioni, per esempio dal nodo $n_1$[a] abbiamo la mappatura verso $a_1, a_2$[b] distinguendo tra le osservazioni $z_0, z_1$.
Assumiamo che $\mathcal{M}$ in Figura 2.4 sia nello stato $s_1$ e che $\mathcal{A}$ nel nodo $n_1$, sappiamo che $z_0 := O(s_1)$ e $\sigma_{\mathcal{A}}$ sceglie come azione $a_1$ con probabilità

$$\gamma(n_1, z_0)(a_1) = 0.5,$$

infine diciamo che con probabilità 0.7 $\mathcal{M}$ evolve nello stato $s_2$.
Dunque, $FSC\mathcal{A}$ aggiorna il nodo di memoria con: $\delta(n_1, z_0, a_1)(n_1) = 0.5$, per stare in $n_1$.
Possiamo dire che semanticamente la transizione: $\langle s_1, n_1 \rangle \to \langle s_2, n_1 \rangle$ ha probabilità $0.5 \cdot 0.5 \cdot 0.7 = 0.175$.
La derivazione di questo valore è stato ottenuto usando la formula:
$p \cdot q_1 \cdot 0.7$ dove: p si riferisce alla parametrizzazione delle due azioni possibili del POMDP $\{\alpha_1, \alpha_2\}$, mentre $q_1$ si riferisce alla generazione del parametro che tiene conto del nodo di riferimento.

[a] Omettiamo la parte per il nodo $n_2$, in quanto dovremmo generare tutte le transizioni anche per quest'ultimo.

[b] Rappresentano gli stati di memoria.

In generale possiamo per ogni POMDP, FSC dato, possiamo generare i valori delle transizioni della Figura 2.6.

| **Act** | $\mathcal{P}$ | **Nodo** | **Result** |
|---|---|---|---|
| $\alpha_1 : p$ | 0.3 | $n_1 : q_1$ | $0.3 \cdot p \cdot q_1$ |
| | | $n_2 : 1 - q_1$ | $0.3 \cdot p \cdot (1 - q_1)$ |
| | 0.7 | $n_1 : q_1$ | $0.7 \cdot p \cdot q_1$ |
| | | $n_2 : 1 - q_1$ | $0.7 \cdot p \cdot (1 - q_1)$ |
| $\alpha_2 : 1 - p$ | 0.5 | $n_1 : q_1$ | $0.5 \cdot (1 - p) \cdot q_1$ |
| | | $n_2 : 1 - q_1$ | $0.5 \cdot (1 - p) \cdot (1 - q_1)$ |
| | 0.5 | $n_1 : q_1$ | $0.5 \cdot (1 - p) \cdot q_1$ |
| | | $n_2 : 1 - q_1$ | $0.5 \cdot (1 - p) \cdot (1 - q_1)$ |

Tabella 2.1: Tabella delle possibili transizioni.

# Capitolo 3

# Sintesi parametrica

## 3.1 Regione

Iniziamo questo capitolo introducendo il concetto di *regione*. Questo concetto è fondamentale per le definizioni sottostanti e si lega al concetto di *istanziazione*, come descritto in sottosezione 2.4.2.

Considereremo soltanto vincoli polinomiali del tipo $g \sim 0$, con $g \in C[\mathcal{X}]$ e dove $\sim \in \{<, \leq, >, \geq\}$.

Diciamo quindi regione $R$ su $\mathcal{X}$ per il quale esiste C(R) $\subseteq$ C$[\mathcal{X}]$ come un insieme di vincoli polinomiali soggetto alle loro congiunzioni:

$$\Phi(R) = \wedge_{c \in C(R)}\ c,$$

$$\text{abbiamo } R = \{val \mid \Phi(R)[val]\}.$$

$C(R)$ viene anche chiamata *rappresentazione di R*.

Ad esempio [5], se definiamo $V = \{p, q\}$ e abbiamo una rappresentazione del tipo $C(R) = \{p^2 + q^2 - 1 \leq 0,\ p + q - 1 \leq 0\}$.
Andiamo a calcolare $R$, quindi abbiamo:

$$R = \{val \mid (p^2 + q^2 - 1)[val] \leq 0,\ (p + q - 1)[val] \leq 0\}$$

e ci chiediamo se per un set di valori sia vera o meno l'istanza.
Prendiamo un set di valori, per esempio: $val_1 = (\frac{2}{5}, \frac{3}{5})$ e $val_2 = (\frac{1}{2}, \frac{4}{5})$.
Verifichiamo che, sostituendo i valori di $val_1$, abbiamo:

$$(\frac{2}{5})^2 + (\frac{3}{5})^2 - 1 \leq 0 \quad \frac{2}{5} + \frac{3}{5} - 1 \leq 0$$

Quindi certamente $val_1$ è istanza di $R$, mentre $val_2$ non lo è, in quanto soddisfa la prima disuguaglianza, ma non la seconda:

$$\frac{1}{2} + \frac{4}{5} - 1 = \frac{3}{10} > 0$$

## 3.2 Problemi della sintesi parametrica

Ora introduciamo il problema della *sintesi parametrica*, in particolar modo, definiremo il concetto di *specifica* e cercheremo di definire la computabilità in parallelo a come abbiamo definito la nozione di raggiungibilità come nel sottosezione 2.2.2.

Procediamo sviluppando il significato al concetto di Specifica e soddisfacibilità, quindi dato un pMC $D$ su $\mathcal{X} = \{x_1, ..., x_n\}$, una specifica $\varphi$ e una regione $R \subseteq \mathbb{R}^n$.

Diciamo anche $val \in R$ una valutazione $v : \mathcal{X} \to \mathbb{R}$ con $v(x_i) \in R|i$ per tutti gli $i$:

$$D, R \models \varphi \text{ se e solo se } (\forall val \in R.\ D[v] \models \varphi)$$

Stiamo fissando che dato un pMC e data una Regione, allora possiamo dire che una valore di valutazione val di questa regione può rendere vera la specifica $\varphi$.

Qual'è il motivo del perchè ci possa interessare il fatto che valutazioni soddisfino o meno una determinata specifica?

Il fatto di considerare sia le soluzioni che soddisfano le condizioni sia quelle che non le soddisfano ha portato alla nascita di approcci diversi per arrivare a una soluzione. Possiamo dividere in tre categorie:

Feasability problem: $\exists v \in R.\ D[v] \models \varphi$

Optimal Feasability problem: cercare un'istanza $v \in R$ che massimizza o minimizza

Region Verification: $D, R \models \varphi$ or $D, R \models \neg\varphi$ or $D, R \not\models \varphi \wedge\ D, R \not\models \neg\varphi$

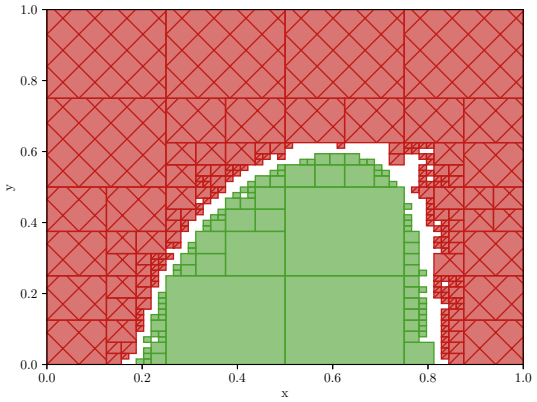



Figura 3.1: Parameter space partitioning dove abbiamo le valutazioni accettate che sono colorati di verde, le valutazione non accettate colore rosso [4].

## 3.3 Feasability problem

Ora invece ci concentreremo su questo tipo di problema che non è banale e che come vedremo non sempre è possibile trovare una valutazione di v che soddisfi una specifica.

Definiamo ora l'*ETR-SAT Decision problem* consiste nel trovare una formula quanficabile $\exists x_1, ..., x_n.\ F(x_1, ..., x_n)$ tale per cui rendere vere le componenti che abbiamo in gioco, dove $F$ è una funzione *booleana* di polinomi su $x_1, ..., x_n$.

Il motivo del perchè abbiamo dovuto l'*ETR-SAT* è in quanto la decision problem si classifica, in termini di Complessità computazionale, all'interno della classe *ETR*.

In particolar modo possiamo completare dando un esempio in questi termini.

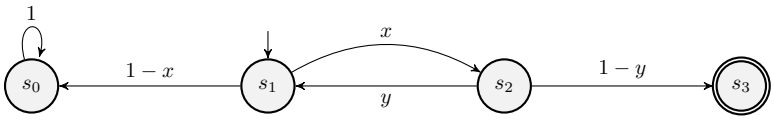



Figura 3.2: Esempio di pMC per raggiunbilitià di probabilità.

Fase di Encoding feasability come ETR-SAT:
Dato un *pMC* D come in Figura 3.2, con regione
$R = \{v \mid 0 < val(x), val(y) < 1\}$ e target $= \{s_3\}$.
Vogliamo rispondere alla domanda: Esiste un'istanza che soddisfi
$(\geq 3/4)$?

$$\exists p_0, \ldots, p_3, x, y.\ 0 < x < 1 \ \wedge\ 0 < y < 1 \ \wedge\ p_1 \geq \frac{3}{4}$$
$$\wedge\ p_3 = 1 \ \wedge\ p_0 = 0$$
$$\wedge\ p_2 = yp_1 + (1-y)p_3$$

In particolare possiamo dire che le variabili $p_0, p_1, p_2, p_3$ sono definite per ogni stato $s_i$ di D e rappresentano la probabilità di raggiungere lo stato $s_3$.
$x$ e $y$ rappresentano i parametri di D.
Rispetto alla formulazione abbiamo una prima parte dove vengono definite regione e specifica, la seconda che setta i valori per gli stati per il quale sicuramente raggiungiamo $s_3$ e per il quale non lo raggiungiamo $s_0$.
L'ultima parte si riferisce ad un equazione ricorsiva per le probabilità raggiunbili, viene riformulata anche come:

$$p_s := \sum_{s' \in S_{>0}} P(s, s') \cdot p_{s'} \quad \text{per tutti gli } s \in S_{>0} \setminus G$$

dove $S_{>0} = \{s_1, s_2, s_3\}$ è l'insieme dei possibili stati che possono raggiungere gli stati target in $G$.

### 3.3.1 Algoritmo di eliminazione degli stati

Data l'enorme difficoltà nel computare i risultati che vogliamo ottenere, l'*eliminazione di Gauss* risulta parecchio scarsa in termini di prestazioni.

Vengono quindi definiti due algoritmi specifici per cercare di eliminare il maggior numero di stati così da portare in computazione un modello sempre più piccolo.

La *state elimination* si basa su due algoritmi fondamentali:

(1) *Adding short-cut*: L'algoritmo da come si può vedere in Figura 3.3 dato uno stato s che deve raggiungere lo stato finale $t$, rimpiazza tutte le possibili generando una scorciatoia tenendo conto delle transizioni rimpiazzate.

Rimane di fatto una sola e unica transizione che porta ad uno stato e quindi con questa riduzione non abbiamo modo di raggiungere lo stato passando per esempio da altri.

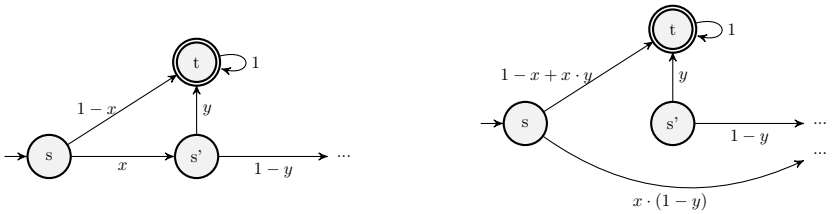



Figura 3.3: Adding short-cut.

(2) *Self-loop removal*: Invece la *eliminating self-loops* si occupa invece di eliminare i self-loop sugli stati, ovviamente andando a tenerne conto nelle rispettive transizioni in uscita per i prossimi stati.




Figura 3.4: Self-loop removal.

## 3.4 Discesa del gradiente

Eccoci finalmente arrivati alla definizione del metodo usato per tutta l'analisi svolta nella fase di sperimentazione. In particolar modo ci soffermeremo non su gli algoritmi in se, poichè c'è ne sono più di uno, ma ci focalizziamo su come agisce e quali sono le idee alla base.

### 3.4.1 Introduzione

La *Discesa del gradiente* o GD è una tecnica largamente usata per massimizzare/minimizzare una funzione obiettivo $f[u]$.

Chiaramente vengono effettuati degli aggiornamenti rispetto ai pesi che nel nostro caso sono i nostri parametri, così da poterci muovere all'interno della nostra funzione rispetto al suo gradiente in questo caso $\partial_p f[u]$. Per comodità rispetto agli esempi portati in merito a questo capitolo, useremo il concetto di gradiente come la sola e unica derivata parziale di $f[u]$ rispetto a p, proprio perchè è l'unico parametro che abbiamo in Figura 3.5.

In generale però non sempre abbiamo modelli di questo tipo quindi in generale quanto si valuta il gradiente di una funzione data, lo si valuta rispetto a tutte le sue componenti, quindi:

$$\nabla f[u] = \begin{bmatrix} \frac{\partial f}{\partial p_1}, \\ \\ \frac{\partial f}{\partial p_2}, \\ \\ \vdots, \\ \\ \frac{\partial f}{\partial p_n} \end{bmatrix}^{1} \quad \text{dove} \quad p_1, \ldots, p_n \text{ sono tutti parametri del modello.}$$

Ora possiamo risolvere il problema dato nella Capitolo 3, ovvero cercare di usare questo algoritmo per trovare un istanza $u \in R$ dove $ER^{s_0 \to T'}[u] \sim \lambda$

L'unico algoritmo che tratteremo sarà il *Plain GD* che è uno dei più semplici e sta alla base di qualsiasi altro algoritmo successivo.

Tendenzialmente questi algoritmi hanno idea alla base ma cambiano in termini di prestazioni computazionali, ovvero gli algoritmi sucessivi al Plain GD godono di proprietà in più oppure di termini aggiuntivi che servono per migliorarne le prestazioni.

Questo esempio semplificato di pesudocodice ci permette di capire a grandi linee il funzionamento del gradient discent.

Inoltre notiamo che si fa riferimento al GD-Method, un metodo che ci permette di settare rispetto alla nuova formulazione trovata.

[1] In questo esempio ho descritto il gradiente di $f[u]$ in formato vettoriale con le sue componenti, i parametri.

**Algorithm 1** GD.

```
1: while f[u] ≤ λ do
2:     if u è un ottimo locale then
3:         Trova un'istanza di u
4:     end if
5:     Aggiorna u con il GD-Method
6: end while
7: return u
```

Infine diciamo che gli algoritmi come questo di base ha una variabile da settare tra cui il *batch-size*, quest'ultimo regola quante variabile vengono aggiornate rispetto all'istanza u trovata.

### 3.4.2 Algoritmo Plain GD

Il *Plain GD* è il più semplice algoritmo implmentato tra l'altro contenuto all'interno di Storm, viene richiesto soltanto una variaibile, ovvero il *learning rate* $\eta$[2] che regola gli step necessari durante il computo del massimo/minimo (locale).

Ora definiamo in generale la formulazione:

$$u_i^{t+1} = u_i^t + \eta \cdot \partial_{p_i} f[u_i^t]$$

dove $u_i^t$ è il valore di $p_i$ con istanza $u^t$

Questa formulazione generalizza il concetto di istanza futura e attuale, in particolare l'istanza futura la si genera rispetto a quella attuale sommando con il gradiente trovato.

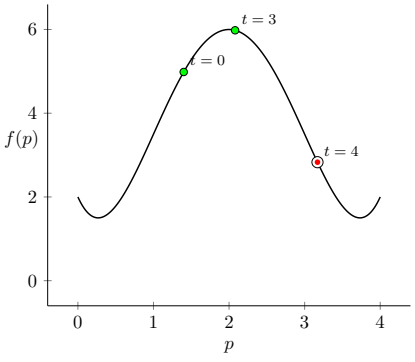



Figura 3.5: Plain GD.

[2] $\eta$ può assuemere un valore compreso tra (0, 1), solitamente viene settato di default 0.1.

Riprendendo l'esempio precedente con il suo grafico in Figura 3.5 possiamo andare a capire come l'algoritmo si è comportato rispetto alla funzione definita come $f(p) = \frac{1}{2}p^4 - 4p^3 + 9p^2 - 4p + 2$ con regione $R = [0, 3]$.
Assumiamo che l'istanza iniziale $u^0(p) = 1$, un learning $\eta = 0.1$ e $\lambda = 5.9$.
Allora rispetto a questa valutazione possiamo notare come il Plain GD si comporta in particolare abbiamo al $t = 0$ dove non abbiamo un massimo locale che invece abbiamo in $t = 3$.
Infine abbiamo il al tempo $t = 4$ (l'ultima iterazione) una valutazione che va oltre il $\lambda$ stabilitio, quindi la valutazione o istanza dell'iterazione precedente era il massimo.

# Capitolo 4

# Strumenti

## 4.1 Prism

Prism è uno strumento frequentemente usato nell'ambito del Model Checking e permette di effettuare numerose analisi con modelli diversi.

Tra i principali modelli ricordiamo: Catene di Markov discrete e continue, Processi decisionali di Markov ad osservazione parziale e addirittura giochi stocastici.

Infine, Prism viene continuamente mantenuto in sviluppo da un team di ricercatori dell'Univerisità di Oxford, nonostante il progetto fosse stato iniziato presso l'Univerisità di Birmingham.

### 4.1.1 Sintassi

La sintassi di Prism non è particolarmente complicata, in quanto povera di costrutti e delimitatori, ovvero non contiene graffe e definizioni avanzate di costrutti.

### 4.1.2 Dichiarazione del modello

Innanzitutto bisogna chiarire il modello che si vuole creare, esistono diversi modelli come abbiamo già descritto in precedenza e ognuno di essi ha una propria sigla, in particolare:

modelli di Markov e modelli di Markov parametrici(DTMC)

modelli decisionali di Markov (MDP)

Processi di Markov continui (CTMC)

processi decisionali di Markov con osservazione parziale (POMDP)

Una volta identificato il modello, siamo in grado definire gli eventuali costrutti. La sintassi di base prevede: variabili, transizioni e valori di probabilità, eventualmente arricchiti con costrutti specifici per il modello scelto.

### 4.1.3 Costrutti di base

Illustriamo ora i moduli, che possono essere intrepretati come funzioni, dove all'interno racchiudono il funzionamento vitale del modello stesso.

Internamente contiene di conseguenza la definizione in termini di transizioni e stati del modello.

Ovviamente, è possibile che all'interno ci sia più di un modulo; in questo modo, si può suddividere e permettere di distinguere tra diverse componenti che si collegano in un unico modello.

```
1  dtmc
2
3  module Esempio
4  x : [0..2] init 0;
5
6
7  [] x=0 -> 0.8 : (x'=1) + 0.2 : (x'=2);
8  [] x=1 -> 1.0 : (x'=0);
9  [] x=2 -> 1.0 : (x'=1);
10
11 endmodule
```

Listing 4.1: Esempio di modello DTMC.

In questo esempio Listing 4.1, notiamo che il modello analizzato presenta una variabile $x$ che assume valori nel range dichiarato `[0, 1]` e ha come stato iniziale 0.

Notiamo anche elencati per ogni stato le possibili transizioni, per esempio `[] x=1 -> 1.0 : (x' = 0)`, ovvero sono nello stato $x = 1$ e se eseguo la transizione ho $\frac{1}{10}$ di possibilità di ritornare nello stato $x = 0$ oppure $\frac{9}{10}$ di possibilità di rimanere nello stesso stato.

In sintesi, possiamo interpretare le transizioni in una maniera generica del tipo:

`[] guardia -> probabilità : aggioramento.`

Infine, l'ultimo costrutto di base è il *reward* e può essere definito all'interno del modulo con la seguente sintassi:

`rewards guardia : assegnazione reward endrewards.`

Ora cerchiamo di fornire un'idea di come viene dichiarato un MDP e di quali sono le caratteristiche principali della sintassi in Prism.

```
mdp

module Esempio
x : [0..2] init 0;


[] x=0 -> 0.8 : (x'=1) + 0.2 : (x'=2);
[] x=1 -> 1.0 : (x'=0);
[] x=2 -> 1.0 : (x'=1);


[a1] x=0 -> 0.6 : (x'=1) + 0.4 : (x'=2);
[a2] x=1 -> 1.0 : (x'=2);
[a3] x=2 -> 0.5 : (x'=0) + 0.5 : (x'=1);

endmodule
```

Listing 4.2: Esempio di modello MDP.

In questo modello, ad esempio, oltre a contenere le classiche transizioni di un DTMC, contiene anche le transizioni etichettate con dei nomi che rappresentano le azioni. La transizione `[a1] x=0 -> 0.6 : (x' = 1) + 0.4 : (x'= 2)` si riferisce ad un azione etichettata come `a1`.

### 4.1.4 Principali modelli analizzati

Concentriamoci ora su due modelli che abbiamo analizzato, di cui uno in particolare è quello utilizzato nell'analisi. Inizieremo dal `PMC` per poi arrivare al `POMDP`.

```
dtmc

const double x;
const double y;

module main

s : [0..2] init 0;

[] (s = 0) -> x : (s' = 1) + 1-x : (s' = 0);
[] (s = 1) -> y : (s' = 1) + 1-y : (s' = 2);
[] (s = 2) -> true;

end module
```

Listing 4.3: Esempio di modello pMC.

Ecco un esempio di un modello base di `pMC`. Analizzando la prima transizione `[] (s = 0) -> x : (s' = 1) + 1-x : (s' = 0);`, possiamo notare che non abbiamo più un valore reale, bensì una variabile.

Questo non ci permette di discutere della transizione in termini di un valore di probabilità fissato a priori, ma richiede l'uso di programmi a supporto di questi tipi di modelli per cercare una soluzione.

Infine, notiamo che deve essere fatta una dichiarazione precedente dei parametri che etichettano transizioni definite, esattamente come delle comuni variabili.

Nella mia trattazione però, il modello pMC non viene trattato come modello in linguaggio Prism, perchè la traduzione fatta da Storm produce un file DRN che è un formato completamente diverso da quest'ultimo ed è molto difficile da analizzare.

Dedichiamoci ora a definire l'ultimo modello analizzato, partendo da un'astrazione di un problema, ovvero abbiamo pensato di analizzare un agente che si muove all'interno di una griglia 2D.

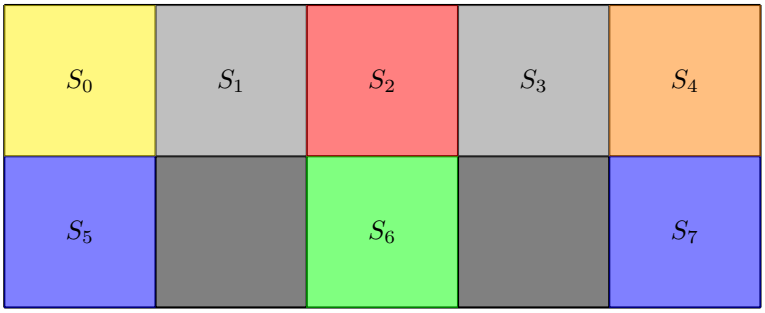



Figura 4.1: Griglia 2D dove l'agente può muoversi liberamente rispetto agli stati dichiarati. Possiamo notare la presenza di due quadrati neri, che come il resto del contorno della griglia, costituiscono i muri della griglia stessa.

Vediamo adesso la rappresentazione in termini sintattici del modello PRISM, generato a partire da questa idea di griglia.

```
1  pomdp
2
3
4  observables
5  o
6  endobservables
7
8
9  module maze
10
11 s : [0..7] init 0;
12 o : [1..6] init 1;
13
14 [east]s=0->1/2:(s'=1)&(o'=2)+1/2:(s'=5)&(o'=5);
15 [west]s=0->1/2:(s'=1)&(o'=2)+1/2:(s'=5)&(o'=5);
16 [north]s=0->1/2:(s'=5)&(o'=5)+1/2:(s'=1)&(o'=2);
17 [south]s=0->1/2:(s'=5)&(o'=5)+1/2:(s'=1)&(o'=2);
18
19 [east]s=1->1/2:(s'=2)&(o'=3)+1/2:(s'=0)&(o'=1);
```

```
[west]s=1->1/2:(s'=0)&(o'=1)+1/2:(s'=2)&(o'=3);
[north]s=1->1/2:(s'=2)&(o'=3)+1/2:(s'=0)&(o'=1);
[south]s=1->1/2:(s'=2)&(o'=3)+1/2:(s'=0)&(o'=1);

[east]s=2->2/3:(s'=3)&(o'=2)
+1/6:(s'=1)&(o'=2)+1/6:(s'=6)&(o'=6);
[west]s=2->1/6:(s'=3)&(o'=2)
+2/3:(s'=1)&(o'=2)+1/6:(s'=6)&(o'=6);
[north]s=2->1/3:(s'=3)&(o'=2)
+1/3:(s'=1)&(o'=2)+1/3:(s'=6)&(o'=6);
[south]s=2->1/6:(s'=3)&(o'=2)
+1/6:(s'=1)&(o'=2)+2/3:(s'=6)&(o'=6);

[east]s=3->1/2:(s'=4)&(o'=4)+1/2:(s'=2)&(o'=3);
[west]s=3->1/2:(s'=2)&(o'=3)+1/2:(s'=4)&(o'=4);
[north]s=3->1/2:(s'=2)&(o'=3)+1/2:(s'=4)&(o'=4);
[south]s=3->1/2:(s'=4)&(o'=4)+1/2:(s'=2)&(o'=3);

[east]s=4->1/2:(s'=3)&(o'=2)+1/2:(s'=7)&(o'=5);
[west]s=4->1/2:(s'=3)&(o'=2)+1/2:(s'=7)&(o'=5);
[north]s=4->1/2:(s'=7)&(o'=5)+1/2:(s'=3)&(o'=2);
[south]s=4->1/2:(s'=7)&(o'=5)+1/2:(s'=3)&(o'=2);

[east]s=5->(s'=0)&(o'=1);
[west]s=5->(s'=0)&(o'=1);
[north]s=5->(s'=0)&(o'=1);
[south]s=5->(s'=0)&(o'=1);

[east]s=7->(s'=4)&(o'=4);
[west]s=7->(s'=4)&(o'=4);
[north]s=7->(s'=4)&(o'=4);
[south]s=7->(s'=4)&(o'=4);

[done]s=6->true;

endmodule

label "target"=o=6;
label "bad"=o=5;
```

Listing 4.4: Modello POMDP della griglia precedentemente descritta.

Rispetto al modello precedente, possiamo notare che gli stati di osservazioni sono numerati e per ognuno di essi c'è uno stato di osservazione. Ad esempio:
`o=1 - (s0) stato identificato dal colore giallo`
`o=2 - (s1 , s3) stato identificato dal colore grigio`
`o=3 - (s2) stato identificato dal colore rosso`
`o=4 - (s4) stato identificato dal colore arancione`
`o=5 - Bad (s5, s7) stato identificato dal colore blu`
`o=6 - Target (s6) stato identificato dal colore verde`

Notiamo subito che nelle prime righe è presente una dichiarazione importante, ovvero `observables ... endobservables`, che permette di definire le variabili destinate a tenere conto delle osservazioni.

Inoltre, queste variabili devono essere dichiarate precedentemente, esattamente come altre comuni variabili, in particolare per esempio noi abbiamo `o : [1..6] init 1`, ovvero esattamente 6 stati di osservazione, molto meno rispetto agli stati dichiarati in `s`, in quanto esistono osservazioni che legano più stati.

La restante formulazione tiene soltanto conto non solo delle eventuali transizioni, quindi cambi di stato ma anche cambi di osservazioni.

Ad esempio, nella transizione `[ east ] s =0 - >1/2:( s '=1) &( o '=2) +1/2:( s '=5) &( o '=5);` ci dice che quando mi trovo nello stato $s_0$ ed eseguo l'azione etichettata come `[ east ]` posso finire con probabilità $\frac{1}{2}$ in $s_1$ solo che devo aggiornare lo stato di osservazione con `(o' =2)`.

Infine la definizione di due label, ovvero `label "target" =o=6;` e `label "bad" =o=5;` che ci permette di identificare degli stati, in particolare diciamo che lo stato target è $s_2$, in quanto $s_2$ ha come stato di osservazione `o = 6`.

Mentre come stati di bad abbiamo $s_5$ e $s_7$ che invece condividono lo stato di osservazione `o = 5`.

## 4.2 Storm

Storm è uno strumento molto usato per effettuare analisi di sistemi, quindi modelli di catene di Markov continue o discrete.

L'unica differenza, rispetto a Prism, è che non permette di generare modelli, il suo unico obiettivo è quello di analizzarli.

Storm accetta in input diverse categorie di formati diversi per la rappresentazione di un modello, in particolare abbiamo formati del tipo JANI, PRISM, DRN, in particolare il formato DRN viene generato dallo strumento stesso.

Infine, Storm viene mantenuto in sviluppo da un team di ricercatori dell'università Tecnica di Aquisgrana.

### 4.2.1 Storm CLI e Stormpy

Storm è interamente sviluppato in linguaggio C++, creato sulla base di quest'ultimo e di specifiche librerie che usa, gode di algoritmi e strutture dati estramamente ottimizzati.

Il team di sviluppo, comprendendo l'enorme difficoltà, soprattutto per i novizi, di usare questo applicativo basato su un linguaggio come C++, ha preferito creare un'interfaccia in Python.

Andando a definire codici e librerie che richiamano sostanzialmente le precedenti librerie sottostanti già definite.

Questa maniera di mascherare, permette l'avvicinamento di nuovi utilizzatori, perchè come sappiamo Python vanta di semplicità in termini di scrittura di codici e ha un enorme portabilità.

Oltre alla definizione di una libreria per la generazione di script utili all'analisi dei modelli, è stata sviluppata un'interfaccia a riga di comando (CLI - Command-Line Interface).

Questa CLI consente di eseguire le analisi direttamente da terminale, permettendo di selezionare i singoli parametri e impostando automaticamente alcuni valori utili per l'analisi. Questo approccio è particolarmente utile per la generazione di script, in cui ogni metodo richiede la corretta parametrizzazione.

### 4.2.2 Motori

Storm supporta circa *engines*[1], progettati per offrire vantaggi specifici in base ai modelli forniti come input per l'analisi. Questi vantaggi riguardano principalmente l'efficienza computazionale e i tempi di esecuzione.

Questi motori non si possono confrontare proprio perchè sono stati concepiti per modelli tra di loro diversi e infatti ognuno di essi viene impiegato a seconda delle circostanza.

Elenchiamo i principali:

Sparse Engine, questo corrisponde al motore principale e viene solitamente usato per modelli con tanti parametri, si appoggia a strutture dati come bitVector e Matrix Sparses. Il fatto di usare queste strutture dati permette una prestazione computazionale molto efficace in termini di memoria, mentre un aumento di tempo in fase di caricamento del modello.

[1] Anche chiamati motori di calcolo.

DD, questo tipo di motore supporta la BDD[2], una struttura dati usata per le rappresentazioni di funzioni booleane. Questo tipo di motore offre vantaggi in termini di caricamento del modello ma svantaggi in termini di efficacia legate alla computazione del modello durante l'analisi.

Hybrid, che è la combinazione dei due motori precendenti. In particolare viene caricato un formato *DD-Based*, che riduce le tempisitiche legate alla fase di build, successivamente vengono effettuate traduzioni dal modello BDD in Sparses Matrix che permettono il continuo dell'analisi sul motore Sparse.

Tutte le analisi compiute nel Capitolo 5 viene fatta usando Sparse Engine, proprio perchè ottimizza meglio per analisi dove i modelli sono parametrici e quindi dove è importante avere strutture dati adeguate per effettuare calcoli complessi velocemente.

### 4.2.3 Script per l'Analisi Automatica con Storm.

Passiamo alla parte pratica dello studio condotto, dove andiamo a effettuare verifiche su proprietà che vogliamo verificare.

In particolare useremo uno script, realizzato in Python, che è diviso in due parti: la prima si concentra sulla build e traduzioni di un modello POMDP e la seconda sull'analisi del modello tradotto in PMC.

**Codice 1: Importazione delle librerie.**

```
import stormpy
import stormpy.info
import stormpy.dft as dft

import pycarl

import stormpy.examples
import stormpy.examples.files

import stormpy.pomdp

import stormpy._config as config

import stormpy.pars as sp
from pycarl.formula import FormulaType, Relation
if stormpy.info.storm_ratfunc_use_cln():
import pycarl.cln.formula
else:
import pycarl.gmp.formula
```

[2]Binary Decision Diagram.

**Codice 2: Parsificazione del modello POMDP e settaggi.**

```
1 pycarl.clear_pools() # oppure clear_variable_pool()
2 path = "/home/paolo/Desktop/tesi/files/models/prova.
      prism"
3 prism_program = stormpy.parse_prism_program(path)
4 threshold = t
5 operator = '>'
6 fsc_k = k
7 formula_str = "P{}{}[!\"bad\"U\"target\"]". format(
      operator, threshold)
8 properties = stormpy.
      parse_properties_for_prism_program(formula_str,
      prism_program)
```

In questo frammento di codice viene eseguita il caricamento del modello, in particolare stiamo usando la funzione di libreria `stormpy.parse_prism_program(path)` che permette il caricamento del modello scritto in linguaggio Prism.

Inoltre vengono effettuate le parsificazioni delle proprietà legate al modello e settata la specifica, ad esempio nella riga 11 vediamo l'invocazione della funzione per la parsificazione della proprietà.

`parse_properties_for_prism_program` necessita di due parametri, la formula in formato stringa e il precedente modello caricato.

**Codice 3: Build del modello e semplificazione.**

```
1 pomdp = stormpy.build_model(prism_program, properties)
2 pomdp = stormpy.pomdp.make_canonic(pomdp)
3 pomdp = stormpy.pomdp.make_simple(pomdp)
```

La build del modello viene eseguita solo dopo la aver eseguito la parsificazione del modello e delle sue proprietà, successivamente vengono eseguite due funzioni in merito a rendere il modello canonico e ridotto.

La riduzione serve soltanto ha eliminare degli stati che secondo la sua analisi potrebbero risultare ridondanti o addirittura inutili.

**Codice 4: Traduzione del modello.**

```
1 memory_builder = stormpy.pomdp.PomdpMemoryBuilder()
2 memory = memory_builder.build(stormpy.pomdp.
      PomdpMemoryPattern.selective_counter,fsc_k)
3 pomdp = stormpy.pomdp.unfold_memory(pomdp, memory)
4
5 pmc = stormpy.pomdp.apply_unknown_fsc(pomdp, stormpy.
      pomdp.PomdpFscApplicationMode.simple_linear)
6 print(pmc)
```

In questo blocco di codice invece andiamo a effettuare la traduzione, in particolare viene usato `stormpy.pomdp.PomdpMemoryBuilder()` per tenere conto della memoria del POMDP, successivamente lo andiamo a buildare rispetto a quanti gradi di memoria `fsc_k` abbiamo dichiarato.

Infine andiamo a effettuare a fare l'unfold della memoria ed a effettuare la traduzione con la funzione dichiarata alla riga 5, questo metodo genera un PMC.

**Codice 5: Estrazione del modello generato dalla traduzione in drn.**

```
1 export_options = stormpy.core.DirectEncodingOptions()
2 export_options.allow_placeholders = True
3 stormpy.export_to_drn(pmc, "test.out")
```

Viene effettuata l'estrazione del modello tradotto in formato DRN, questo perchè ora passiamo all'analisi da Storm CLI.

Inoltre, diciamo che il formato DRN, non è un linguaggio come Prism ma una rappresentazione che probabilmente gli sviluppatori di Storm hanno ritenuto utile per la generazione del file tradotto.

La differenza con Prism sta nel fatto che esso gode di una sua sintattica e semantica e quindi permette l'interpretazione del modello generato.

Invece il formato DRN è sempre leggibile ma non ha una sintattica, semantica deducibile e quindi difficilmente interpretabile.

Ora invece trattiamo quello che è l'invocazione di Storm da linea di comando e dei vari settaggi.

Listing 4.5: Esempio di invocazione tramite CLI di Storm.

```
1  Storm-pars 1.8.2 (dev)
2  DEBUG BUILD
3
4  Date: Thu Oct 10 15:45:57 2024
5  Command line arguments:
6  --mode feasibility
7  '--feasibility:method' gd
8  --explicit-drn /home/paolo/Desktop/tesi/files/
       scripts/test.out
9  --prop 'P<0.8[!"bad"U"target"]'
10 --gradient-descent
11 --learning-rate 0.1
12 --precision 0.1
13 --descent-method plain
```

A primo impatto possiamo notare che l'invocazione tiene conto di tanti parametri, alcuni però posso non essere esplicitati perchè vengono inizializzati di default.

I primi due sono i più importanti in quanto detengono il settaggio principale di quello che si vuole analizzare, abbiamo `--mode` che può assumere valori del tipo `feasability, partitioning, verification`.

Quindi come possiamo dichiarare differenti tipi di analisi esattamente come le avevamo categorizzate nella parte di sintesi parametrica.

Selezionando `feasability` siamo automaticamente vincolati a definire `method: pla o gd`, pla sarebba la Parameter lifting area mentre Gd si riferisce al Gradient discent.

Gli altri due parametri obbligatori sono: la specifica o proprietà che in questo caso sarebbe `--prop 'P<0.8[!"bad"U"target"]'` e il modello, in questo caso abbiamo proprio `--explicit-drn` con il suo path.

Successivamente abbiamo dei parametri che solitamente non si settano, il primo è il learning rate, viene settato solitamente a 0.1 di default e per il genere di modelli che trattiamo, ovvero con tanti parametri, è un parametro corretto.

Abbiamo anche `--precision 0.1` che viene di norma settato a 0.1 e ci permette di decidere il grado di precisione decimale.

Infine, abbiamo `--descent-method plain` , il settaggio dell'algoritmo di gradient discent da usare, noi dobbiamo settarlo questo parametro perchè di base viene settato quello computazionalmente più vantaggio ovvero, l'Adam (Adaptive Moment Estimation) [3].

Noi invece avendolo trattato nella maniera più semplice e non avendo necessariamente di chissà quale sforzo computazionale, lo settiamo `method Plain`.

Ora invece esaminiamo con più chiarezza i log relativi a quello che ci restituisce Storm.

L'output generalmente impiega sia la parte legata allo script, dove abbiamo delle informazioni da terminale legate alla traduzione e alla stampa di modelli, sia la parte legata all'analisi usando lo strumento.

Listing 4.6: Output di build del modello e riduzione dello stesso.

```
Time for model construction: 0.063s.

------------------------------
Model type:     DTMC (sparse)
States:         42
Transitions:    83
Reward Models:  none
State Labels:   3 labels
* target -> 1 item(s)
* bad -> 6 item(s)
* init -> 1 item(s)
Choice Labels:  none
------------------------------

Time for model simplification: 0.010s.
------------------------------

Model type:     DTMC (sparse)
States:         17
Transitions:    34
Reward Models:  none
State Labels:   4 labels
* bad -> 1 item(s)
* init -> 1 item(s)
* target -> 1 item(s)
* _target -> 1 item(s)
Choice Labels:  none
------------------------------
```

Notiamo subito che Storm, per ogni modello, stampa tutte le caratteristiche tipiche del modello stesso, tra le principali abbiamo: il tipo, gli stati e le transizioni.

Inoltre rispetto a questra prima fase, abbiamo la costruzione del modello e successivamente la fase di semplificazione o riduzione.

Infine, possiamo notare che per ogni operazione abbiamo una stampa delle tempistiche in termini di tempo.

Listing 4.7: Output dell'analisi svolta da Storm.

```
1  Find feasible solution for
2  P=? [(!"bad") U "target"]
3  Parameters p15_0 p14_0 p9_0 p8_0 p7_0 p2_0 p6_0
       are inconsequential. They will be omitted in
       the found instantiation.
4  Finding an extremum using Gradient Descent
5  Trying out a new starting point
6  Trying initial guess (p->0.5 for every parameter
       p or set start point)
7
8  Starting at {
9  p12_0 : 4503608634569751/9007199254740992,
10 p13_0 : 4503608634569751/9007199254740992,
11 p0_0 : 4503608634569751/9007199254740992,
12 p1_0 : 4503608634569751/9007199254740992,
13 p3_0 : 4503608634569751/9007199254740992,
14 p5_0 : 4503608634569751/9007199254740992}
```

Dopo aver buildato e semplifcato il modello, passiamo all'analisi vera e propria.

Storm per come lo avevamo invocato, inizializza tutte i coefficienti dei parametri a $\frac{1}{2}$ e cerca un valore tale che soddisfi il valore di soglia precisato nella specifica.

Possiamo notare come la traduzione abbiamo di fatto trovato questo insieme di parametri, sono tutti generati e numerati in maniera automatica e quindi molto meno interpretabili.

Listing 4.8: Output dell'analisi svolta da Storm.

```
Aborting because the bound is satisfied

Result at initial state: 0.2017976214 ( approx.
    0.2017976214) at

[p12_0=4503608634569751/9007199254740992,
p13_0=284772190865662467/576460752303423488,
p0_0=4503608634569751/9007199254740992,
p1_0=4503608634569751/9007199254740992,
p3_0=28131342201323541/576460752303423488,
p5_0=4503608634569751/9007199254740992].
Time for model checking: 0.028s.

Performance statistics:
* peak memory usage: 139MB
* CPU time: 0.159s
* wallclock time: 0.132s
```

Infine abbiamo la parte di output finale dove abbiamo questa dicitura `Aborting because the bound is satisfied` che ci dice che l'algoritmo è terminato.

Possiamo notare che è riuscito a trovare un valore soddisfacente per la specifica analizzata.

Viene anche stampato a video non solo il valore trovato ma anche l'insieme dei possibili valori trovati per ogni parametro.

# Capitolo 5

# Sperimentazione e Conclusioni

## 5.1 Casi di studio

Concludiamo con l'analisi dei casi di studio, in particolare ci soffermiamo su alcuni risultati che verrano riportati successivamente in tabelle.

Ovviamente il modello sulla quale si fa riferimento è quello descritto in sottosezione 4.1.4, mentre per la parte di analisi si fa riferimento sottosezione 4.2.3.

La seguente Tabella 5.1, riporta i risultati della specifica $P_{>0.7}(\neg B\ U\ G)$ dove esattamente come in Figura 4.1, abbiamo $B = \{s_5, s_7\}$ e $G = \{s_6\}$ .

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **iterazione** |
|---|---|---|---|---|---|
| $P > 0.7$ | 0.2694779039 | 1 | 17 | 34 | 1 |
| | 0.2695897224 | | | | 2 |
| | 0.2698472731 | | | | 5 |
| | 0.2678534185 | 2 | 52 | 172 | 1 |
| | 0.2684364827 | | | | 2 |
| | 0.269599765 | | | | 8 |

Tabella 5.1: Valori analizzati con $P > 0.7$.

Possiamo notare come rispetto ai dati trovati in Tabella 5.1, l'algoritmo fatica a trovare dei valori che soddisfino la specifica data, risulta evidente infatti che la probabilità a cui tenda sia di al più circa 0.27.

In queste tabelle inoltre, viene tenuto conto dell'*iterazione*, ovvero quante volte l'algoritmo ha cercato di ricalcolare provando a istanziare i parametri in maniera diversa.

Infatti possiamo notare il confronto tra il valore trovato dopo sette iterazioni a partire dalla prima, dove si vede un miglioramento, in particolare sembra che il valore aumenti.

Caso interessante anche è il confronto tra gli ultimi valori al variare di $k$, possiamo notare che in questo caso, nonostante abbiamo aumentato $k$ il valo-

re ottenuto è comunque migliore di quello ottenuto nel caso di modello senza memoria.

Concentriamoci ora sulla valutazione della proprietà seguente: $P_{>0.2}(\neg B\ U\ G)$, dato che la precedente tabella ha prodotto valori che sembrano non superare il valore di 0.27, allora cerchiamo di avvicinarci alla soglia in questione.

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **tempo** |
|---|---|---|---|---|---|
| $P > 0.2$ | 0.2017976214 | 1 | 17 | 34 | 0.112s |
| | 0.2010945557 | 2 | 52 | 172 | 0.674s |
| | 0.2009418181 | 3 | 87 | 108 | 3.595s |
| | 0.2006738534 | 4 | 122 | 444 | 19.958s |
| | 0.2001722883 | 5 | 157 | 580 | 70.404s |
| | 0.2001996378 | 6 | 192 | 716 | 204.637s |
| | 0.2000019494 | 7 | 227 | 852 | 445.076s |
| | 0.2001225851 | 8 | 262 | 988 | 925.412s |
| | 0.2000058304 | 9 | 297 | 1124 | 1976.142s |
| | 0.2000156292 | 10 | 332 | 1260 | 3559.787s |

Tabella 5.2: Valori analizzati per $P > 0.2$.

Possiamo osservare come il valore più vicino alla soglia 0.2 sia proprio il modello con $k = 7$, rispetto a questa analisi concludiamo che non possiamo essere sicuri che aumentando k il valore che otteniamo sia migliore o peggiore.

I valori calcolati da Storm sono quelli trovati alla prima iterazione e si collocano al di sopra della soglia di 0.2, ma non rappresentano esattamente la probabilità di soddisfare la proprietà delle pMC.

Successivamente ci concentriamo sui valori ottenibili per valori minori di un threshold dato.

Questi valori in Tabella 5.3 però a differenza della specifica opposta portano ad una riduzione abbastanza evidente, in particolare abbiamo una differenza alta di circa 0.04.

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **tempo** |
|---|---|---|---|---|---|
| $P < 0.2,\ P < 0.3$ | 0.19999982 | 1 | 17 | 34 | 0.105s |
| | 0.1994498774 | 2 | 52 | 172 | 0.395s |
| | 0.1969084141 | 3 | 87 | 308 | 0.667s |
| | 0.1918727697 | 4 | 122 | 444 | 1.000s |
| | 0.1860528701 | 5 | 157 | 580 | 1.687s |
| | 0.1812070246 | 6 | 192 | 716 | 1.624s |
| | 0.1774414739 | 7 | 227 | 852 | 1.775s |
| | 0.1742608644 | 8 | 262 | 988 | 2.057s |
| | 0.171521831 | 9 | 297 | 1124 | 2.527s |
| | 0.169225574 | 10 | 332 | 1260 | 2.604s |

Tabella 5.3: Valori analizzati con $P < 0.2,\ P < 0.3$.

In modo considerevolmente possiamo dire che a fronte di valori grandi di $k$ quando si cerca di minimizzare rispetto ad una certa specifica, sembra che il

modello tradotto aiuti fortemente a trovare valori sempre più piccoli.

L'analisi è stata eseguite anche per $P < 0.3$ trovando gli stessi risultati.

Procediamo nuovamente con un ulteriore analisi, andando a valutare la specifica $P_{<0.12}(\neg B\ U\ G)$.

Vogliamo analizzare specifiche relative a valori minimi: sappiamo che il modello non riesce a trovare soluzioni al di sotto di una certa soglia, ma desideriamo verificare se aumentando il valore di $k$ sia possibile ottenere un value inferiore.

Se consideriamo la Tabella 5.4 possiamo notare che non esiste soluzione che soddisfi la richiesta, infatti i valori oscillano tra $[0.136, 0.137]$.

Inoltre nonostante gli aumenti considerevoli di $k$ e il tentativo da parte dell'algoritmo di riformulare nuovi parametri di istanziazione, il value ottenuto risulta non concludente.

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **iterazione** |
|---|---|---|---|---|---|
| $P < 0.12$ | 0.1362632064 | 1 | 17 | 34 | 1 |
| | 0.1362003234 | | | | 3 |
| | 0.1360178769 | | | | 4 |
| | 0.1359910037 | | | | 5 |
| | 0.1370698994 | 2 | 52 | 172 | 1 |
| | 0.1369055002 | | | | 2 |
| | 0.13554433 | | | | 3 |
| | 0.1373419438 | 3 | 87 | 308 | 1 |
| | 0.1361946059 | | | | 2 |
| | 0.1361736764 | | | | 3 |
| | 0.1360237696 | | | | 4 |

Tabella 5.4: Valori analizzati con $P < 0.12$.

Un altro caso interessante è se aumentiamo di un decimo la specifica, vedi Tabella 5.5, dove possiamo vedere che otteniamo dei valori che si avvicinano alla tabella precedente ma sembrano migliori, soprattutto le ultime tre iterazioni sembrano scendere al di sotto di 0.136. Possiamo inoltre notare che non variandone il $k$ otteniamo dei risultati migliori o comunque leggermente più bassi della specifica precedente.

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **iterazione** |
|---|---|---|---|---|---|
| $P < 0.13$ | 0.1362632064 | 1 | 17 | 34 | 1 |
| | 0.1362536266 | | | | 4 |
| | 0.1361829752 | | | | 10 |
| | 0.1359378896 | | | | 13 |
| | 0.1359278146 | | | | 25 |
| | 0.1359235683 | | | | 119 |

Tabella 5.5: Valori analizzati con $P < 0.13$.

Infine valutiamo un'ultima specifica, di cui i valori sono contenuti in Tabella 5.6, in cui si osserva come i valori nonostante l'aumento di $k$, non siano particolarmente chiari.

Ad esempio, sempre in questo caso al posto di trovare valori più bassi, come ad esempio nella Tabella 5.3, l'algoritmo cerca di avvicinarsi sempre di più al threshold.

| **Proprietà** | **Value** | $k$ | **stati** | **transizioni** | **tempo** |
|---|---|---|---|---|---|
| $P < 0.175$ | 0.1739786354 | 1 | 17 | 34 | 0.229s |
| | 0.1741299307 | 2 | 52 | 172 | 4.607s |
| | 0.1748043918 | 3 | 87 | 308 | 14.426s |
| | 0.1743094597 | 4 | 122 | 444 | 31.162s |

Tabella 5.6: Valori analizzati con $P < 0.175$.

Terminiamo sottolineando che le analisi condotte sono state eseguite per valori di $k$ sempre compresi tra cinque e dieci, questo perchè non sarebbe computazionalmente possibile valutare modelli con tanti stati e transizioni[1].

Inoltre, sembra che nonostante si aumenti $k$ anche per valori superiori a dieci, i risultati comunque si ripetono e quindi non sempre si ha un miglioramento in termini di ottimizzazione.

Ricordiamo che il modello di partenza del POMDP è di dimensioni ridotte; pertanto, non riusciamo a osservare vantaggi significativi con l'aumento di $k$, rendendo i risultati ottenuti meno interessanti.

[1]Non soltanto un limite per la macchina sulla quale viene analizzato ma abbiamo riscontrato numerose problematiche legate ad errori inevitabili del linguaggio C++ sottostante.

## 5.2 Conclusioni finali

In questa tesi abbiamo affrontanto un studio approfondito delle catene di Markov, dove ci siamo soffermati in particolar modo sull'estensione parametrica e sui processi decisionali con osservazione parziale.

L'analisi compiuta non permette di definire chiaramente una conclusione in termini di interpretazione dei dati ottenuti, ma pone l'attenzione su alcuni comportamenti che noi abbiamo ritenuto molto interessanti rispetto al modello analizzato.

Inoltre, l'analisi condotta ci ha permesso non solo di capire gli aspetti in termini di modelli ma anche gli algoritmi sottostanti, come la discesa del gradiente per la computazione dei risultati ottenuti.

Durante lo studio abbiamo trovato problematiche, legate alla poca documentazione in merito all'implementazione degli script usati e della CLI.

Gradualmente, però, siamo riusciti a superare le difficoltà, riuscendo non solo ad analizzare modelli esemplificativi, ma anche a costruirne uno di dimensioni ridotte.

Segnaliamo anche il fatto che esistono problematiche che non dipendono dall'implementazione del modello e non dipendendono dalle risorse per computare i risultati.

Tra questi abbiamo numerosi errori che si riferiscono a problematiche legate alla allocazione delle risorse per le strutture dati a cui Storm si appoggia quando, una volta effettuta la traduzione del modello, deve andare ad analizzarlo.

L'elaborato prodotto rimane comunque soddisfacente rispetto a quello che si voleva ottenere, sicuramente se ci fosse stato più tempo avremmo avuto modo di completare sopratutto quelle parti che richiedono un approfondimento particolare.

Un possibile approfondimento rispetto a questo elaborato sarebbe quello di creare un modello diverso per cercare di capire se è possibile adattarlo ad un altro problema.

# Bibliografia


[1] V. Forejt et al. ≪Automated Verification Techniques for Probabilistic Systems≫. In: *Formal Methods for Eternal Networked Software Systems (SFM'11)*. A cura di M. Bernardo e V. Issarny. Vol. 6659. LNCS. Springer, 2011, pp. 53–113.

[2] Jonas Fritzsch, Tobias Schmid e Stefan Wagner. ≪Experiences from Large-Scale Model Checking: Verifying a Vehicle Control System with NuSMV≫. In: *14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021*. IEEE, 2021, pp. 372–382. DOI: 10.1109/ICST49551.2021.00049. URL: https://doi.org/10.1109/ICST49551.2021.00049.

[3] Linus Heck et al. ≪Gradient-Descent for Randomized Controllers Under Partial Observability≫. In: *Verification, Model Checking, and Abstract Interpretation - 23rd International Conference, VMCAI 2022, Philadelphia, PA, USA, January 16-18, 2022, Proceedings*. A cura di Bernd Finkbeiner e Thomas Wies. Vol. 13182. Lecture Notes in Computer Science. Springer, 2022, pp. 127–150. DOI: 10.1007/978-3-030-94583-1_7. URL: https://doi.org/10.1007/978-3-030-94583-1%5C_7.

[4] Nils Jansen, Sebastian Junges e Joost-Pieter Katoen. ≪Parameter Synthesis in Markov Models: A Gentle Survey≫. In: *Principles of Systems Design - Essays Dedicated to Thomas A. Henzinger on the Occasion of His 60th Birthday*. A cura di Jean-François Raskin et al. Vol. 13660. Lecture Notes in Computer Science. Springer, 2022, pp. 407–437. DOI: 10.1007/978-3-031-22337-2_20. URL: https://doi.org/10.1007/978-3-031-22337-2%5C_20.

[5] Sebastian Junges. ≪Parameter synthesis in Markov models≫. Tesi di dott. RWTH Aachen University, Germany, 2020. URL: https://publications.rwth-aachen.de/record/783179.

[6] Sebastian Junges et al. ≪Parameter synthesis for Markov models: covering the parameter space≫. In: *Formal Methods Syst. Des.* 62.1 (2024), pp. 181–259. DOI: 10.1007/S10703-023-00442-X. URL: https://doi.org/10.1007/s10703-023-00442-x.

[7] PRISM Model Checker Development Team. *PRISM Model Checker*. Accessed: 2024-10-25. 2024. URL: https://www.prismmodelchecker.org/.

[8] SAFEST Project Team. *SAFEST: A probabilistic risk assessment toolchain*. https://www.safest.dgbtek.com/. Accessed: 2024-10-25. SAFEST Project, 2024.

[9] Storm Development Team. *Storm Model Checker*. `https://www.stormchecker.org/`. Accessed: 2024-10-21. 2024.